# Mini Projet

Application web hautement disponible et pouvant etre mise a l'echelle

**Salim LAKHAL & Nossa IYAMU**

CSC 8604 - Cloud Computing | Telecom SudParis | 2025-2026

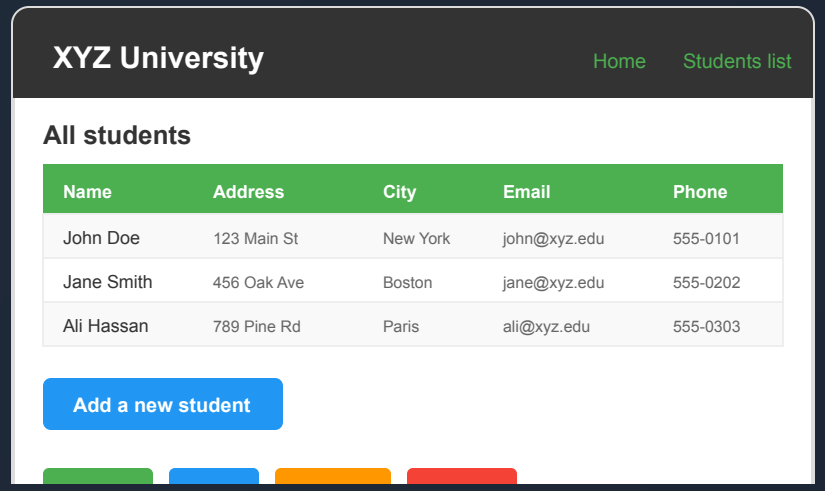
# Scenario

## Le probleme

- L'Universite XYZ gere les dossiers etudiants via une application web
- Pendant les periodes d'admission : **lenteurs** et **indisponibilite**
- Trop de requetes simultanees, un seul serveur ne suffit pas

## Notre mission

- Concevoir une architecture AWS **hautement disponible, scalable et securisee**
- Application CRUD : gestion des dossiers etudiants (nom, adresse, email, telephone)



# Conditions requises par la solution

**Fonctionnelle**
Operations CRUD sans latence perceptible

**Charge equilibree**
Equilibrage du trafic entre serveurs web

**Scalable**
Auto Scaling selon la demande

**Hautement disponible**
Temps d'arret limite avec multi-AZ

**Securisee**
BDD isolee, Secrets Manager, Security Groups

**Couts optimises**
Instances t3.micro, choix d'instances appropriees

# Approche par phases

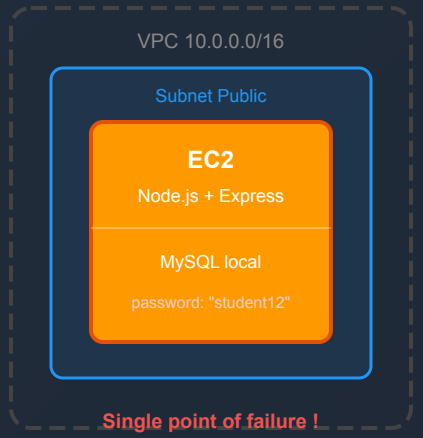
**1 — Planification**
Schema architectural
Estimation des couts

**2 — POC de base**
VPC + EC2 unique
Node.js + MySQL local

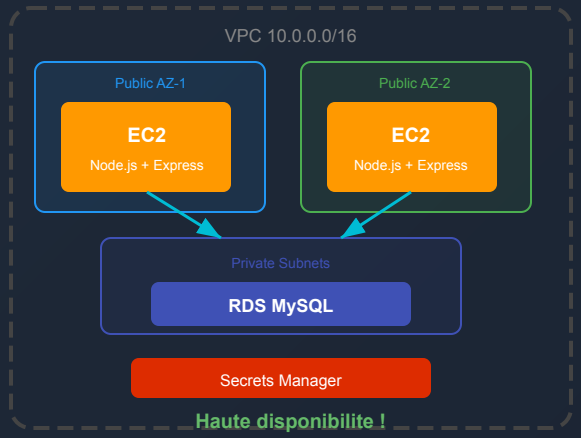
**3 — Decouplage**
RDS + Secrets Manager
IAM Role pour EC2

**4 — HA & Scalabilite**
ALB multi-AZ
Auto Scaling Group

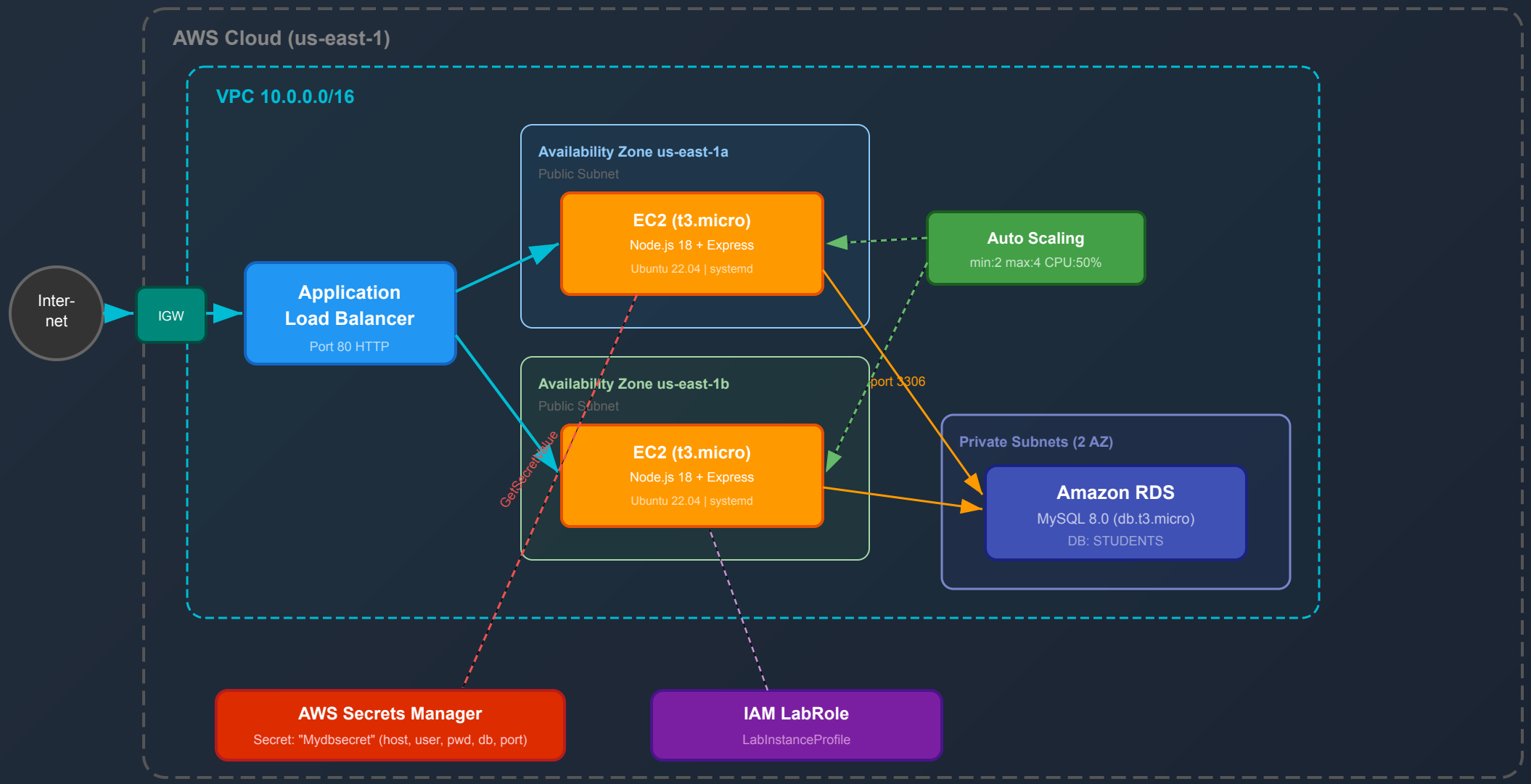
## Phase 2 : Tout sur une machine




## Phase 3-4 : Architecture decouplee



# Architecture finale (Phase 4)



# Services AWS utilises

**VPC** — **Reseau**
VPC, IGW, Subnets (2 publics + 2 prives), Route Tables

**EC2** — **Compute**
t3.micro, Ubuntu 22.04, Node.js 18

**RDS** — **Base de donnees**
MySQL 8.0, db.t3.micro, single-AZ

**ALB** — **Load Balancer**
Application Load Balancer, 2 AZs

**ASG** — **Auto Scaling**
Target Tracking CPU 50%, min:2 max:4

**SM** — **Secrets Manager**
Secret "Mydbsecret" pour les identifiants BDD

**IAM** — **IAM Role**
LabRole + LabInstanceProfile pour EC2

**SSM** — **Systems Manager**
Administration des instances EC2

# Phase 1 & 2 : Planification et POC

### Phase 1 : Planification

- Schema architectural dessine
- Estimation des couts avec AWS Pricing Calculator

### Phase 2 : Application de base

- VPC (10.0.0.0/16) avec sous-reseaux publics et prives
- EC2 t3.micro (Ubuntu) dans le sous-reseau public
- Node.js 18 + Express + MySQL sur une seule instance
- Application CRUD fonctionnelle via IP publique

### Code Phase 2 (credentials en dur)

```
const db = mysql.createConnection({
  host: 'localhost',
  user: 'nodeapp',
  password: 'student12',  // En dur !
  database: 'STUDENTS'
});
```

**Problemes :**

# Phase 3 : Decouplage des composants

## Actions realisees

- Ajout de sous-reseaux prives dans 2 AZs
- Deploiement RDS MySQL dans subnet prive (DB Subnet Group)
- Creation du secret "Mydbsecret" dans Secrets Manager
- IAM Role (LabInstanceProfile) attache a l'EC2
- Security Group RDS : port 3306 uniquement depuis Web SG

## Code Phase 3-4 (Secrets Manager)

```javascript
const { SecretsManagerClient,
  GetSecretValueCommand }
  = require("@aws-sdk/client-secrets-manager");

async function getSecret() {
  const client = new SecretsManagerClient(
    { region: "us-east-1" });
  const response = await client.send(
    new GetSecretValueCommand({
      SecretId: "Mydbsecret"
    }));
  return JSON.parse(response.SecretString);
}
```

# Phase 4 : Haute disponibilite et scalabilite

## Application Load Balancer

- Deploye sur 2 AZs (us-east-1a, 1b)
- Health check sur '/' toutes les 30s
- Point d'entree unique pour l'app

## EC2 Auto Scaling

- Launch Template avec AMI pre-configuree
- ASG : min=2, max=4, desired=2
- Target Tracking : CPU moyen 50%

## Test de charge

- Outil loadtest depuis Cloud9
- Requetes massives sur URL de l'ALB
- Observation du scaling automatique

# Securite de la solution

## Chaine des Security Groups

### Isolation reseau

- BDD dans sous-reseaux prives
- Pas d'acces Internet direct a RDS
- Principe du moindre privilege

### Controle d'acces

- Web SG : ports 80 + 22 uniquement
- DB SG : port 3306 depuis Web SG
- ALB SG : port 80 depuis Internet

### Gestion des secrets

- Secrets Manager pour credentials
- IAM Role pour acces securise
- Zero credentials en dur

# Estimation des couts (12 mois, us-east-1)

| Service | Mensuel | Annuel |
|---|---|---|
| Amazon EC2 (2x t3.micro, On-Demand) | ~$15.33 | ~$184 |
| Amazon RDS MySQL (db.t3.micro, 20 GB) | ~$17.63 | ~$211 |
| Application Load Balancer + LCU | ~$16.20 | ~$200 |
| AWS Secrets Manager | ~$0.40 | ~$5 |
| Data Transfer (10 GB/mois) | ~$0.90 | ~$11 |
| **Total estime** | **~$51/mois** | **~$611/an** |

**Optimisation possible :** Reserved Instances = **-40%** soit ~$367/an

Mini Projet

# Demonstration

Application en live sur AWS

# Enseignements tires

1 **Architecture par phases**
Commencer simple, complexifier progressivement.

2 **Importance du decouplage**
Separation web/BDD = scalabilite + securite.

3 **Secrets Manager vs hardcoding**
Ne jamais mettre les credentials en dur.

4 **Auto Scaling + ALB**
Haute disponibilite automatique sans intervention.

5 **Gestion des couts**
Budget $100, choix d'instances appropriees.

**Ameliorations possibles**

- HTTPS avec certificat SSL (ACM + ALB)
- Multi-AZ RDS pour HA de la BDD
- CloudFront CDN pour assets statiques
- Authentification utilisateur (Cognito)

# Merci !

**Salim LAKHAL & Nossa IYAMU**

CSC 8604 - Cloud Computing | Telecom SudParis | 2025-2026

Questions ?